DevOps Fundamentals - Laboratory Work 1

Linux, Git, GitHub, Docker and Jenkins

Student: Danil Marchenko
Group: IT2-2303
Student ID: 41587
Course: DevOps
Date: 18 October 2026
GitHub repository: https://github.com/jardemAI/DevOps

Student Information

This individual project is stored in `~/devops/marchenko-danil-devops`. It contains the required Linux directory structure, personalized student information, a Bash system script, a Python application and tests, Docker configuration, a Jenkins declarative pipeline, and this report.

Linux

The project includes `app/`, `scripts/`, `config/`, `logs/`, and `backup/`. `Marchenko_Danil_info.txt` contains the student's name, surname, group, ID, and course. `config/app.conf` documents non-secret application settings. The local `logs/application.log` contains 12 records and is excluded from Git; `logs/application.log.example` preserves the sample in the repository.

The following commands create, copy, move, inspect, search, and remove files:

```
pwd
ls -la
cd ~/devops/marchenko-danil-devops
mkdir -p /tmp/devops-demo
touch /tmp/devops-demo/source.txt
cp /tmp/devops-demo/source.txt /tmp/devops-demo/copy.txt
mv /tmp/devops-demo/copy.txt /tmp/devops-demo/renamed.txt
cat Marchenko_Danil_info.txt
head -n 5 logs/application.log
tail -n 5 logs/application.log
rm -r /tmp/devops-demo
find . -name application.log
grep 'ERROR' logs/application.log
grep 'WARNING' logs/application.log
grep -c 'ERROR' logs/application.log
tail -n 5 logs/application.log
```

`pwd` prints the working directory. `ls` lists entries; `cd` changes directory; `mkdir` creates directories; `touch` creates a file or updates its timestamp; `cp` copies; `mv` moves or renames; `rm` removes; `cat` prints file contents; `head` and `tail` print the first and last lines. `find` searches by path or name. `grep` selects matching lines and `grep -c` counts them. In the sample log, the searches returned two `ERROR` records and three `WARNING` records.

`private.txt` is mode `600` (`rw-------`), so only the owner can read and write it. `public.txt` is mode `644` (`rw-r--r--`), so the owner can read and write and the group and others can read. `r` means read, `w` means write, and `x` means execute for a file or directory traversal for a directory. The owner is the file's user, the group is its assigned group, and others are all remaining users. Git does not preserve ordinary read/write differences; after checkout, use `chmod 600 private.txt && chmod 644 public.txt`. The Bash script is executable.

`./scripts/Marchenko_Danil_system.sh` ran successfully. It displayed student and system information using `whoami`, `hostname`, `date`, `uname`, `df`, and `free`, and confirmed that the student information file exists.

Process management was checked with `ps`, `ps aux`, and `top -b -n 1`. A process is a running program instance; PID is its process identifier. `ps` prints a point-in-time list, while `top` refreshes CPU and memory information interactively. In the captured sample, PID `1`, owned by `root`, was `systemd`, using `0.0%` CPU and `0.1%` memory. These values are snapshots and may differ on another machine.

Git and GitHub

The project was initialized on `main` and configured with the student's name. Its history contains meaningful changes for the project layout and Linux utilities, personalized student information, the Python application and tests, Docker, Jenkins, and a development-branch feature merged into `main`.

Representative commits in the new history are:

```
a0366de Create Linux lab structure and utilities
06a7331 Add personalized Python application and tests
d03823b Add Docker image and Compose configuration
22fb89c Add Jenkins declarative pipeline
fdb0bba Add JSON readiness endpoint on development
fb26430 Merge readiness endpoint into main
```

The `development` branch adds a JSON `/ready` endpoint and an automated test. A feature branch isolates work so it can be tested before merging into the main line. Inspect repository history and remotes with:

```
git status
git log --oneline --graph --all
git branch -av
git remote -v
```

`.gitignore` excludes `*.log`, `backup/`, and `.env`, preventing runtime logs, backup data, and local settings from being committed. A disposable ignored log, environment file, and backup file were used to verify the matching rules with `git status --short --ignored` and `git check-ignore -v`. `backup/.gitkeep` preserves the required empty directory; normal backup files remain ignored.

The new repository is hosted at jardemAI/DevOps. Git records versioned changes and branch merges; GitHub hosts the shared remote copy for collaborators and automation.

Docker

`Dockerfile` includes `FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE`, and `CMD`. It uses a Python slim base image, copies the application into `/app`, runs it as a non-root account, exposes port `8000`, and starts the HTTP server. The application displays personalized student information and provides `/health`, `/student`, `/version`, `/metadata`, and `/ready` routes.

On a Docker-enabled host, build and run the application with:

```
docker build -t marchenko-danil-devops .
docker images
docker run -d --name marchenko-danil-container \
  -p 8000:8000 \
  -e STUDENT_NAME=Danil \
  -e STUDENT_SURNAME=Marchenko \
  -e STUDENT_GROUP=IT2-2303 \
  -e STUDENT_ID=41587 \
  marchenko-danil-devops
docker ps
docker ps -a
docker logs marchenko-danil-container
curl http://localhost:8000/health
curl http://localhost:8000/ready
```

A Docker image is a packaged template; a container is a running or stopped instance created from that image. Environment variables are named runtime settings that let one image be reused with different values. Secrets should not be hard-coded into an image because they can remain in image layers or registries; supply them through an approved runtime secret mechanism.

`docker-compose.yml` defines the image, port, environment, and container name. Use `docker compose up --build`, inspect services with `docker compose ps`, and remove them with `docker compose down`.

Execution limitation: Docker is not installed in this sandbox (`docker: command not found`). Therefore no Docker image or container was built here. Docker and Compose commands must still be run on a Docker-enabled host; the configuration is prepared but is not represented as a successful build.

Jenkins

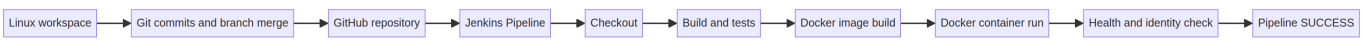
The `Jenkinsfile` defines declarative stages `Checkout`, `Build`, `Test`, `Docker Build`, and `Docker Run`. It supplies the student environment variables, runs the Python tests, builds an image, launches a temporary container, checks the HTTP response, prints the container logs, and removes the container after the run.

Create a Jenkins Pipeline job named `Marchenko_Danil_DevOps`. Configure **Pipeline script from SCM**, Git repository `https://github.com/jardemAI/DevOps`, branch `*/main`, and script path `Jenkinsfile`. The repository is public, so checkout does not need credentials. The Jenkins agent requires Python 3 and Docker Engine access. Configure a trigger only after Jenkins is reachable by GitHub.

When started, Jenkins retrieves the Jenkinsfile and source from GitHub, assigns an agent, runs the stages in order, and records the console output and result. If a stage fails, later stages normally do not execute and Jenkins marks the build failed unless the pipeline handles the error. Fix the cause before rerunning.

Execution limitation: No Jenkins server URL or job access was supplied, and the sandbox has no Docker executable. The Jenkins job was not created or run. There is no genuine Jenkins dashboard, stage view, console log, image, container, or `Finished: SUCCESS` result to show. Do not treat the Jenkinsfile itself as proof of a successful pipeline.

Integrated DevOps Workflow



Linux provides the shell and operating-system tools used to create and inspect project files. Git records changes and merges. GitHub hosts the repository and provides the source Jenkins checks out. Jenkins automates the build and tests, then uses Docker to package and start the application. A health and student-information check verifies the launched application. The image is the reusable package; the container is its runtime instance.

If a pipeline stage fails, later stages normally do not execute. Jenkins records a failed build, and the cause must be fixed before the next run. This workflow turns a local project into a versioned, remotely available, and automatable build.

Execution Evidence

The Python unit tests, Bash system-information script, and HTTP endpoint checks passed in the sandbox. Jenkins-specific screenshots and console output are not included because no accessible Jenkins instance was supplied. Capture authentic Jenkins evidence after running the configured job; no successful remote run is claimed in this report.

Conclusion

The Linux structure and command demonstrations, file permissions, log searches, Git history and merge, `.gitignore`, public GitHub repository, Python application, tests, Docker configuration, Compose file, Jenkinsfile, workflow diagram, and this report are prepared. Five Python unit tests passed. A real Docker build and run and a Jenkins job and pipeline still need to be executed on an accessible host.